Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Departamento de Ciencias Computacionales



“Sprint 1. Del dato horario al indicador diario”

Para la materia:
Análisis y visualización de la información – 2026B

Presenta
Manzo Mejía Susana Rubí - 216819441
Castellanos Zuñiga Ian Michael - 220122358
Montero González José Antonio - 223386879
Melendez González Ángel Leonel - 220963603
Anaya Valladolid Luis Rafael - 213330824
Magaña Mojica Jesús Ismael - 218217759

Carrera
Ingeniería Informática

Profesor:
MC. Nayeli Areli Pérez Padilla

Guadalajara, Jalisco, México; 28 de septiembre de 2026

Introducción

El monitoreo constante de la calidad del aire representa un pilar fundamental para la salud pública y la gestión ambiental. El presente reporte documenta el análisis exhaustivo de los datos atmosféricos recabados durante el mes de marzo del año 2024 por la estación de monitoreo Centro (CEN), perteneciente al Sistema de Monitoreo Atmosférico de Jalisco (SIMAJ). El objetivo central de este sprint es transformar las mediciones físicas en crudo en información estratégica de riesgo, aplicando rigurosamente los lineamientos dictados por la NOM-172-SEMARNAT-2023. A través de la evaluación de indicadores dinámicos, se busca identificar los patrones de contaminación, determinar cuáles son los agentes dominantes y establecer recomendaciones operativas basadas en la evidencia técnica para la protección ciudadana.

Desarrollo de la Actividad

Estación: SIMAJ (CENTRO).

Fuente: Sistema de Monitoreo Atmosférico de Jalisco.

Descripción y preparación de los datos

Para este proyecto se trabajó con los datos correspondientes a marzo de 2024 de la estación de monitoreo CEN (Centro), ubicada en Churubusco No. 143, Centro de Salud No. 1, en el Sector Libertad de Guadalajara. La información se extrajo de la base histórica general del Sistema de Monitoreo Atmosférico de Jalisco en formato BD_2024.xlsx, filtrando únicamente los registros del mes y la estación asignados al equipo en esta caso marzo 2024, se elaboró un archivo llamado CEN_marzo.xlsx con 744 registros, correspondientes a los 31 días del mes multiplicados por 24 horas.

Antes de cualquier análisis se llevó a cabo una revisión del archivo: se verificó que no existieran filas duplicadas, que la secuencia de fechas y horas fuera continua sin huecos las 744 marcas horarias esperadas estaban presentes, que la columna de hora estuviera en el rango correcto (0 a 23) y que la estación reportada fuera consistentemente "CEN" en todo el archivo. Esta revisión confirmó que el archivo estaba correcto, sin errores que corregir.

Después se documentó, para cada variable disponible, su tipo de dato, su unidad de medición oficial y su rango físico para la zona (por ejemplo, O3 y CO en partes por millón, PM10 y PM2.5 en microgramos por metro cúbico, temperatura en grados Celsius, presión atmosférica en milímetros de mercurio). Esta tabla de referencia permitió, en etapas posteriores, distinguir entre un valor estadísticamente extremo pero físicamente válido y un valor que representará un error de medición.

Se calculó también la disponibilidad de información por variable y en general: de los 744 registros esperados, la estación CEN presentó una disponibilidad alta (superior al 98%) en O3, CO, PM10, PM2.5, temperatura, dirección del viento y presión atmosférica. Sin embargo, se identificó que las variables de humedad relativa (RH) y velocidad del viento (WS), junto con precipitación, radiación solar e índice UV, se encuentran completamente vacías (744 de 744 registros faltantes) durante todo el mes. Esta ausencia se investigó de manera puntual y se determinó que corresponde a una limitación estructural del instrumento en esa estación no a un error de captura del archivo ni a un hueco temporal recuperable por lo que se documenta como una limitación conocida del set de datos, en lugar de intentar completarla con datos falsos.

Criterios de limpieza y transformación

El criterio general adoptado para la limpieza de los datos fue el de no eliminar ni modificar valores sin evidencia de que se trataran de errores, siguiendo la instrucción explícita de distinguir entre un posible error de sensor y un evento ambiental real antes de tomar cualquier decisión.

Para ello, se aplicó un doble filtro sobre cada variable: primero, se comparó cada valor contra el rango físico documentado previamente (por ejemplo, que la temperatura no excediera límites razonables para la zona, o que la dirección del viento se mantuviera entre 0 y 360 grados); en segundo lugar, se aplicó un criterio estadístico de detección de valores atípicos mediante el rango intercuartílico (regla de 1.5×IQR) para identificar extremos dentro del rango físico que merecieran revisión adicional.

El resultado de este análisis mostró que ningún valor del conjunto de datos rebasó su rango físico plausible, por lo que no se detectaron errores evidentes de sensor. Los valores

señalados como atípicos por el criterio estadístico se examinaron según su patrón temporal antes de decidir qué hacer con ellos:

* Los atípicos en CO, PM10 y PM2.5 se concentraron sistemáticamente en el horario de 6:00 a 9:00 de la mañana, repitiéndose de manera consistente a lo largo de numerosos días del mes. Este patrón es compatible con el pico de tráfico vehicular matutino combinado con la inversión térmica característica de esas horas en la Zona Metropolitana de Guadalajara, por lo que se interpretaron como eventos reales y recurrentes, y se conservaron sin modificación.
* Un bloque de valores inusualmente bajos en la variable de temperatura, concentrado en dos periodos continuos de aproximadamente 24 a 30 horas cada uno (6-7 y 12-13 de marzo), no siguió el mismo patrón diario recurrente que los casos anteriores. Al no contar con evidencia que confirmara si se trató de un evento meteorológico real o de una falla temporal del sensor, este bloque se marcó para revisión, pero se mantuvo dentro del conjunto de datos sin eliminarlo, dado que no existe justificación suficiente para descartarlo.

Con base en este análisis, el criterio final adoptado fue conservar la totalidad de los registros originales, documentando explícitamente tanto la ausencia estructural de RH y WS como el bloque de temperatura señalado para revisión, en lugar de aplicar imputaciones, interpolaciones o eliminaciones que pudieran introducir sesgos no justificados en el análisis posterior.

Para la etapa de transformación, los datos horarios validados se procesaron conforme a los criterios de la NOM-172-SEMARNAT-2023 para obtener los indicadores del Índice AIRE Y SALUD: promedio móvil de 8 horas para CO, promedio móvil ponderado de 12 horas (NowCast) para PM10 y PM2.5, y el valor horario directo para O3, clasificando cada resultado en las categorías correspondientes (Buena, Aceptable, Mala, Muy Mala, Extremadamente Mala) según las tablas vigentes a partir de enero de 2024.

Aplicación de la NOM-172-SEMARNAT-2023

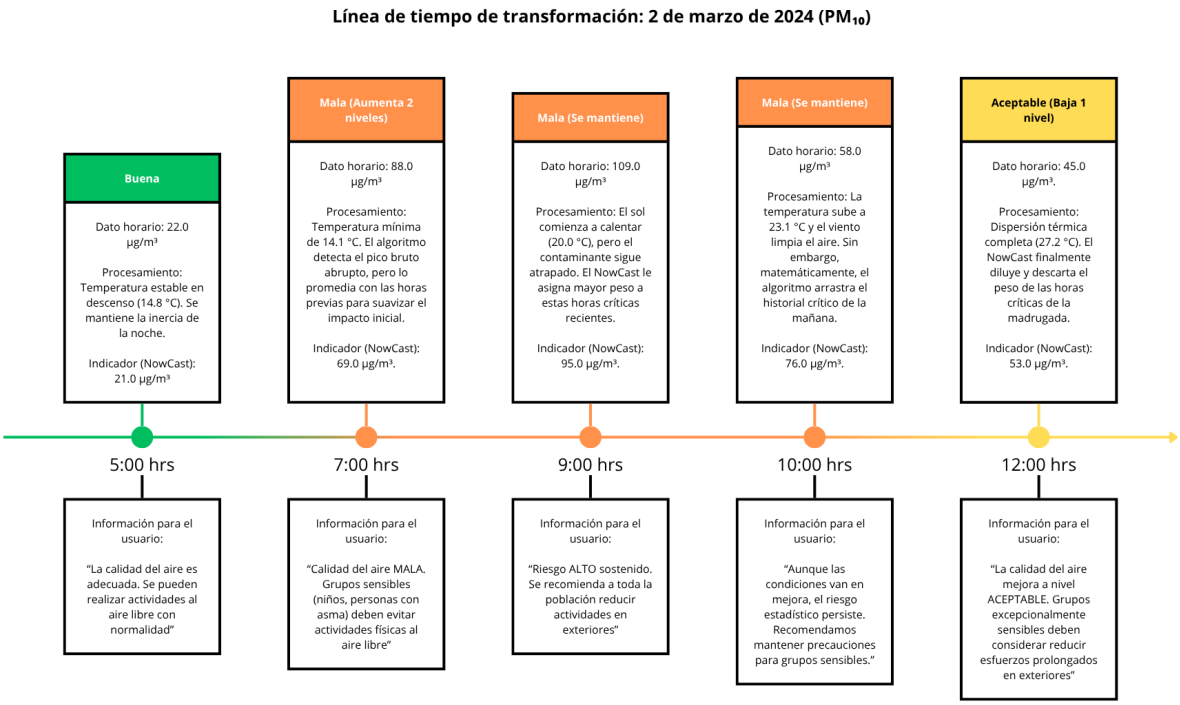
El procesamiento de los datos se apegó estrictamente a los lineamientos establecidos por la NOM-172-SEMARNAT-2023, aplicando una diferenciación metodológica obligatoria entre el análisis horario y la agregación diaria.

Para el cálculo diario, el procesamiento se ajustó a lo estipulado en la sección de Concentraciones base para el cálculo del Índice Aire y Salud para cada contaminante. El algoritmo NowCast que se utilizó en la evolución horaria fue sustituido por el promedio aritmético de 24 horas para las partículas suspendidas (PM_{10} y $PM_{2.5}$). Para la representatividad estadística de este promedio, se implementó la regla de suficiencia de datos del 75%, la cual exige un mínimo de 18 lecturas horarias válidas por jornada. En el caso de los gases (O_3 y CO), el indicador diario se obtuvo extrayendo el valor máximo de sus respectivos promedios móviles u horarios registrados durante el día.

Previo a la asignación de las categorías de riesgo, los resultados numéricos se sometieron a los criterios de redondeo exigidos por la normativa. Posteriormente, los valores obtenidos fueron evaluados utilizando exclusivamente los umbrales más estrictos (algunos vigentes a partir de enero de 2024), haciendo un énfasis especial en los límites de frontera. Destaca una de las actualizaciones del umbral para la categoría “Aceptable” de las partículas suspendidas PM_{10}, el cual se redujo a 60 $\mu g/m^3$.

Por último, para determinar la Categoría Global del Índice Aire y Salud, tanto en el análisis horario como en el diario, se aplicó el principio normativo de máximo riesgo. El algoritmo se encargó de evaluar de manera simultánea todos los contaminantes disponibles y le asignó al periodo la categoría correspondiente al contaminante que presentara el nivel de riesgo más alto, etiquetándolo como el contaminante dominante o responsable de dicha ventana de tiempo.

Simulación de la información horaria



- ¿Qué información estaba disponible en cada momento?

  Para cada hora de la simulación se contaba de manera simultánea con la concentración física instantánea (el dato crudo del PM_{10}), las condiciones meteorológicas (temperatura del ambiente) y el cálculo retrospectivo del algoritmo NowCast. Al realizar la evaluación del valor crudo y el valor arrojado por el NowCast nos permitió observar la diferencia entre un pico aislado de contaminación y el riesgo de exposición sostenida para la salud.

- ¿Cuándo se presentaron los cambios más importantes?

  El primer cambio crítico que pudimos evaluar ocurrió a las 7:00 hrs, esto cuando la acumulación sostenida de partículas empujó el indicador oficial a cruzar el umbral hacia la categoría Mala (aumentando dos niveles de golpe). Después, el siguiente momento clave se suscitó a las 10:00 hrs, donde se evidenció el rezago matemático que presenta el NowCast, esto porque la contaminación real disminuyó de una manera abrupta casi por mitad, pero el indicador oficial se mantuvo con la alerta activa debido a las condiciones severas de las horas previas.

- ¿Qué variables pudieron relacionarse con estos cambios?

  Un factor físico determinante fue la temperatura exterior. A las 7:00 hrs se registró la temperatura mínima de la mañana (14.1 °C), lo que facilitó una capa de inversión térmica que atrapó los contaminantes a nivel del suelo, pero a partir de las 10:00 hrs, el abrupto incremento de la temperatura (hasta 23.1 °C) ocasionó que está capa estancada se rompiera, ocasionando que la atmósfera dispersara las partículas acumuladas.

- ¿Qué debería comunicar la agencia en ese momento?

  A las 7:00 hrs, nuestra agencia debe emitir una alerta preventiva indicando a los grupos sensibles (niños, adultos mayores, personas con afecciones respiratorias) el evitar actividades físicas y prolongadas al aire libre. A partir de las 10:00 hrs, aunque el ambiente y el cielo se perciban más limpios, la alerta debe mantenerse activa por norma, comunicando que el riesgo estadístico continúa. Para las 12:00 hrs, la agencia puede proceder a comunicar de manera oficial el levantamiento de las restricciones preventivas al consolidarse la tendencia del contaminante a la baja.

Numeralia

Análisis del mes de Marzo

Estación: Centro (CEN)

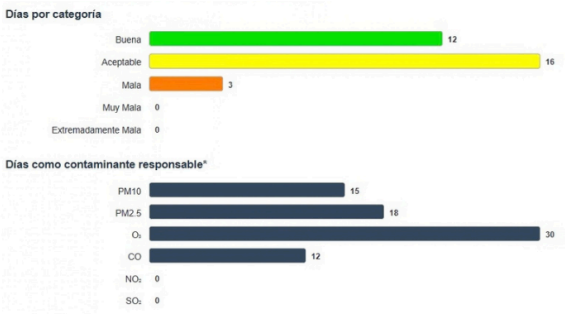
Días cubiertos: 31 días

Categoría predominante: Aceptable

Contaminante principal: O_3 (Ozono)



*La categoría diaria representa al peor estado registrado en la jornada.



*Los empates de categoría global cuentan para cada contaminante responsable; los conteos pueden sumar más de 31.

Días críticos

10 de Marzo | **12 de Marzo** | **28 de Marzo**

Los días que alcanzaron el nivel más alto registrado en el mes, llegando a la categoría “Mala”.

Calendario de resumen diario



¿Qué categoría predominó?

Aceptable, en 16 de 31 días. Hubo 12 días Buena y 3 Mala.

¿Qué contaminante fue dominante con mayor frecuencia?

O_3, presente como dominante en 30 días. Los empates cuentan para cada contaminante, por eso los conteos pueden sumar más de 31.

¿Qué diferencias existen entre la interpretación horaria y la diaria?

La horaria permite ver a qué hora sube o baja el indicador y cuándo cambia la categoría; la diaria resume cada contaminante con su indicador del día y muestra la peor categoría resultante. Las gráficas actuales muestran el resumen diario, no la evolución por hora.

¿Qué información se pierde al resumir todo un día en una sola categoría?

La hora y duración de los episodios, los cambios de categoría, los picos breves y si la contaminación se mantuvo elevada o solo subió por un periodo corto. También se pierde el detalle horario de cómo coincidieron los contaminantes y las condiciones meteorológicas.

Principales hallazgos

El análisis exhaustivo de los datos recabados durante marzo de 2024 revela patrones atmosféricos relevantes que requieren monitoreo continuo:

En primer lugar, destaca la dominancia del gas Ozono (O_3) como contaminante principal, definiendo la categoría diaria en 30 de los 31 días evaluados. Este comportamiento indica que, durante la transición hacia la primavera, la alta radiación solar y el incremento de temperaturas en la zona Centro aceleran la reacción fotoquímica de los gases precursores. Como resultado, la calidad del aire se estancó de manera persistente en el nivel "Aceptable".

Por otro lado, el análisis horario permitió identificar un comportamiento focalizado de las partículas suspendidas (PM_{10} y $PM_{2.5}$). Se observó que los picos críticos de concentración están fuertemente condicionados por las variables meteorológicas matutinas, específicamente por el descenso de temperatura que favorece la formación de inversiones térmicas. Si bien estas partículas no dominaron la estadística del mes completo, sí actuaron como el detonante principal para alcanzar la categoría "Mala" durante los días de mayor riesgo ambiental (10, 12 y 28 de marzo).

Interpretación desde la perspectiva de la agencia

Tomando en cuenta los hallazgos técnicos obtenidos en la estación Centro, nuestra agencia ha determinado las siguientes directrices operativas y de comunicación para la gestión del riesgo ambiental:

- Focalización de recomendaciones por horario

  Dado que el Ozono (O_3) es el contaminante que domina en la zona y su concentración máxima coincide con los periodos en los que se tiene mayor radiación solar, el SIMAJ deberá emitir recomendaciones formales a la Secretaría de Educación, a las instituciones deportivas y a la ciudadanía en general para que las actividades al aire libre sean programadas y se efectúen preferentemente en el transcurso matutino (antes del mediodía). Con esta medida operativa, buscaremos reducir significativamente la exposición de la población en los momentos de mayor riesgo fisiológico.

- Alertamiento preventivo basado en meteorología

  Tras haber comprobado que el indicador de partículas suspendidas (PM_{10} y $PM_{2.5}$) escala a una categoría “Mala” impulsado por las bajas temperaturas matutinas junto con la falta de dispersión, la agencia buscará integrar estrechamente el monitoreo atmosférico con los pronósticos del clima. Esto permitirá anticipar madrugadas excepcionalmente frías que sean propensas a inversiones térmicas para programar avisos precautorios preventivos desde la noche anterior para proteger en mejor medida a los grupos más vulnerables.

- Priorización de la comunicación horaria

  El análisis realizado a los datos crudos nos demuestra que asignar una sola categoría global a un día completo puede ocultar pequeñas ventanas de tiempo donde la calidad del aire es segura (“Buena”). Por lo tanto, la agencia deberá fortalecer la difusión del Índice Aire y Salud a través de más plataformas, para educar a la ciudadanía para que su toma de decisiones a la hora de realizar actividades al aire libre se base en las lecturas horarias y no en los boletines diarios.

Conclusiones y limitaciones

- **Conclusiones**

  El procesamiento de la información obtenida de la estación CEN demuestra que el comportamiento atmosférico de la zona Centro durante el mes de marzo del año 2024 estuvo regido por dos dinámicas muy diferenciadas. Por un lado, se observó una persistencia crónica del gas Ozono (O_3), cuya reacción fotoquímica ocasionó que la calidad del aire se estancara de manera general durante el mes en la categoría “Aceptable”. Por otro lado, los episodios esporádicos agudos de riesgo ambiental (categoría “Mala”) fueron detonados principalmente por la acumulación de partículas suspendidas (PM_{10} y $PM_{2.5}$) durante las primeras horas del día, esto debido a las inversiones térmicas y el tráfico vehicular matutino.

  Además, este ejercicio comprueba de manera técnica la importancia de mantener una comunicación horaria con la población, ya que basar las alertas en un promedio de 24 horas puede ocasionar que se ignoren episodios críticos.

- **Limitantes**

  Para garantizar la mayor objetividad técnica en la realización de este reporte, es necesario que se establezcan las siguientes restricciones en el análisis:

  1. Anomalías térmicas no concluyentes: Se detectaron un par de bloques de valores inusualmente bajos en la variable de temperatura durante dos períodos casi continuos que iban del 6 al 7 y del 12 al 13 de marzo. Al carecer de los registros meteorológicos adicionales en la misma estación para confirmar si se trató de un evento climático atípico o una falla temporal de los sistemas, los datos se conservaron para proteger la integridad de la información, pero su interpretación debe manejarse con cautela.
  2. Alcance micro-local: Los hallazgos reportados, especialmente en la dominancia crónica del O_3, reflejan de manera exclusiva las condiciones atmosféricas del sector visualizado por la estación Centro. Dado que la dispersión de los contaminantes varía radicalmente según la geografía y las barreras urbanas, este diagnóstico no debe extrapolarse como un panorama definitivo para toda la Zona Metropolitana de Guadalajara.

Código o evidencia técnica empleada

- Para las evaluaciones y cálculos horarios, se empleó el siguiente código:

```
import pandas as pd
import numpy as np

df = pd.read_csv('CEN_marzo.csv')
df.columns = [c.strip() for c in df.columns]
df['DATE'] = pd.to_datetime(df['DATE'])
df = df.sort_values(['DATE']).reset_index(drop=True)
df.head()
def nowcast_una_ventana(valores):
    valores = pd.Series(valores, dtype='float64')

    # Requisito de completión: al menos 2 de las 3 horas mas recientes
    if valores.iloc[-3:].notna().sum() < 2:
```

```
        return np.nan

    validos = valores.dropna()
    if validos.empty:
        return np.nan

    cmax, cmin = validos.max(), validos.min()
    rango = cmax - cmin

    if cmax == 0:
        w = 1.0
    else:
        w = max(0.5, round(1 - rango / cmax, 2))

    n = len(valores)
    suma_pond, suma_pesos = 0.0, 0.0
    for i, val in enumerate(valores):
        if pd.isna(val):
            continue
        horas_atras = (n - 1) - i
        peso = w ** horas_atras
        suma_pond += val * peso
        suma_pesos += peso

    if suma_pesos == 0:
        return np.nan
    return round(suma_pond / suma_pesos)


def calcular_nowcast(serie, ventana=12):
      """Aplica nowcast_una_ventana de forma deslizante sobre toda la
serie horaria."""
    valores = serie.values
    resultado = [np.nan] * len(valores)
    for i in range(len(valores)):
        if i < ventana - 1:
            continue
         resultado[i] = nowcast_una_ventana(valores[i - ventana + 1: i +
1])
    return pd.Series(resultado, index=serie.index)
```

```
def promedio_movil_co(serie, ventana=8, minimo=6):
    """Promedio movil simple de 8 horas para CO, requiere al menos 6 de 8 horas validas."""
                                return       serie.rolling(window=ventana, min_periods=minimo).mean().round(2)


# --- Tablas de categorias (NOM-172-SEMARNAT-2023, vigentes desde enero 2024) ---

def cat_o3(x):
    if pd.isna(x): return 'NA'
    if x < 0.058: return 'Buena'
    if x <= 0.090: return 'Aceptable'
    if x <= 0.135: return 'Mala'
    if x <= 0.175: return 'Muy Mala'
    return 'Extremadamente Mala'

def cat_co(x):
    if pd.isna(x): return 'NA'
    if x < 5: return 'Buena'
    if x <= 9: return 'Aceptable'
    if x <= 12: return 'Mala'
    if x <= 16: return 'Muy Mala'
    return 'Extremadamente Mala'

def cat_pm10(x):
    if pd.isna(x): return 'NA'
    if x < 45: return 'Buena'
    if x <= 60: return 'Aceptable'
    if x <= 132: return 'Mala'
    if x <= 213: return 'Muy Mala'
    return 'Extremadamente Mala'

def cat_pm25(x):
    if pd.isna(x): return 'NA'
    if x < 15: return 'Buena'
    if x <= 33: return 'Aceptable'
```

```
    if x <= 79: return 'Mala'
    if x <= 130: return 'Muy Mala'
    return 'Extremadamente Mala'

def cat_no2(x):
    if pd.isna(x): return 'NA'
    if x < 0.053: return 'Buena'
    if x <= 0.106: return 'Aceptable'
    if x <= 0.160: return 'Mala'
    if x <= 0.213: return 'Muy Mala'
    return 'Extremadamente Mala'

def cat_so2(x):
    if pd.isna(x): return 'NA'
    if x < 0.035: return 'Buena'
    if x <= 0.075: return 'Aceptable'
    if x <= 0.185: return 'Mala'
    if x <= 0.304: return 'Muy Mala'
    return 'Extremadamente Mala'

RANGO_BANDAS = {'Buena':1, 'Aceptable':2, 'Mala':3, 'Muy Mala':4,
'Extremadamente Mala':5}

def categoria_global(row):
                cats    =    [row['Categoria_O3'],    row['Categoria_CO'],
row['Categoria_PM10'], row['Categoria_PM2.5'],
            row['Categoria_NO2'], row['Categoria_SO2']]
    cats = [c for c in cats if c != 'NA']
    if not cats:
        return 'NA'
    return max(cats, key=lambda c: RANGO_BANDAS[c])
resultado = df[['STATION', 'DATE', 'HOUR']].copy()

# Concentraciones base horarias
resultado['O3'] = df['O3']
resultado['CO_prom_8h'] = promedio_movil_co(df['CO'])
resultado['PM10_NowCast'] = calcular_nowcast(df['PM10'])
resultado['PM2.5_NowCast'] = calcular_nowcast(df['PM2.5'])
resultado['NO2'] = df['NO2'] if 'NO2' in df.columns else np.nan
resultado['SO2'] = df['SO2'] if 'SO2' in df.columns else np.nan
```

```
# Categorias horarias por contaminante
resultado['Categoria_O3'] = resultado['O3'].apply(cat_o3)
resultado['Categoria_CO'] = resultado['CO_prom_8h'].apply(cat_co)
resultado['Categoria_PM10'] = resultado['PM10_NowCast'].apply(cat_pm10)
resultado['Categoria_PM2.5']                                          =
resultado['PM2.5_NowCast'].apply(cat_pm25)
resultado['Categoria_NO2'] = resultado['NO2'].apply(cat_no2)
resultado['Categoria_SO2'] = resultado['SO2'].apply(cat_so2)

# Categoria global de la hora = la mas desfavorable
resultado['Categoria_Global']     =     resultado.apply(categoria_global,
axis=1)

resultado.head(20)
print('Filas totales:', len(resultado))
print()
print('Distribución Categoría Global:')
print(resultado['Categoria_Global'].value_counts())
resultado.to_csv('CEN_marzo_Indice_AIRE_Y_SALUD.csv', index=False)
files.download('CEN_marzo_Indice_AIRE_Y_SALUD.csv')
```

- Para las evaluaciones y cálculos diarios, se empleó el siguiente código:

```
from collections import Counter, defaultdict
from datetime import datetime
from pathlib import Path
import csv
import unicodedata

BASE = Path(__file__).resolve().parent
INPUT_NAMES = ["CEN_marzo.xlsx - CEN_marzo.csv", "CEN_marzo.csv"]
YEAR, MONTH, N_DAYS = 2024, 3, 31
CATEGORIES = ["Buena", "Aceptable", "Mala", "Muy Mala", "Extremadamente
Mala"]
RANK = {name: i for i, name in enumerate(CATEGORIES)}
POLLUTANTS = ["PM10", "PM2.5", "O3", "CO", "NO2", "SO2"]

#  Cortes  de  categoría  aplicables  a  marzo  de  2024  según  la
NOM-172-SEMARNAT-2023.
```

```
BREAKS = {
    "PM10": [45, 60, 132, 213], "PM2.5": [15, 33, 79, 130],
    "O3": [.058, .090, .135, .175], "CO": [5.00, 9.00, 12.00, 16.00],
    "NO2": [.053, .106, .160, .213], "SO2": [.035, .075, .185, .304],
}
METRICS = {
    "PM10": ("PM10", "promedio diario 24 h (µg/m³)"),
    "PM2.5": ("PM2.5", "promedio diario 24 h (µg/m³)"),
    "O3": ("O3", "máximo horario diario (ppm)"),
    "CO": ("CO_prom_8h", "máximo promedio móvil 8 h (ppm)"),
    "NO2": ("NO2", "máximo horario diario (ppm)"),
    "SO2": ("SO2", "máximo horario diario (ppm)"),
}

def find_input():
    for name in INPUT_NAMES:
        p = BASE / name
        if p.exists():
            return p
      candidates = sorted(p for p in BASE.glob("*CEN*marzo*.csv") if
"export" not in p.name.lower())
    if candidates:
        return candidates[0]
           raise FileNotFoundError("No encuentro 'CEN_marzo.xlsx -
CEN_marzo.csv' junto al script.")

def unique_headers(headers):
    seen, out = Counter(), []
    for value in headers:
        name = value.strip()
        seen[name] += 1
        out.append(name if seen[name] == 1 else f"{name}_{seen[name]}")
    return out

def norm(value):
             value = unicodedata.normalize("NFD", str(value or
"").strip().lower())
    return "".join(c for c in value if unicodedata.category(c) != "Mn")

def round_base(pollutant, value):
```

```
    if value is None: return None
    if pollutant in ("PM10", "PM2.5"):
        return int(value + .5)
    places = 2 if pollutant == "CO" else 3
    scale = 10 ** places
    return int(value * scale + .5) / scale

def category_for(pollutant, value):
    value = round_base(pollutant, value)
    if value is None or value < 0: return None
    for i, limit in enumerate(BREAKS[pollutant]):
        if value <= limit:
            return CATEGORIES[i]
    return CATEGORIES[-1]

def number(value):
    if value is None:
        return None
    s = str(value).strip()
    if not s or norm(s) in {"na", "nan", "nd", "none"}:
        return None
     # La fuente CSV guarda los decimales con coma y los cita, p. ej.
"0,025".
    if "," in s and "." not in s:
        s = s.replace(",", ".")
    try:
        return float(s)
    except ValueError:
        return None

def read_source():
    source = find_input()
    with source.open("r", newline="", encoding="utf-8-sig") as f:
        reader = csv.reader(f)
        original_headers = next(reader)
        headers = unique_headers(original_headers)
        required = {"DATE", "HOUR", "PM10", "PM2.5", "O3", "CO_prom_8h"}
        if not required.issubset(headers):
                    raise ValueError(f"Faltan columnas requeridas en
{source.name}: {sorted(required-set(headers))}")
```

```
        records = []
        for row in reader:
            if not row:
                continue
            row += [""] * (len(headers) - len(row))
            rec = dict(zip(headers, row))
            try:
                dt = datetime.fromisoformat(rec["DATE"].strip())
            except (TypeError, ValueError):
                continue
            if dt.year != YEAR or dt.month != MONTH:
                continue
            records.append((dt, rec))
    records.sort(key=lambda x: x[0])
    if not records:
          raise ValueError("El archivo no contiene registros de marzo de
2024.")
    return source, headers, records

def summarize(records, headers):
    by_day = defaultdict(list)
    for dt, row in records:
        by_day[dt.day].append((dt, row))
    out = []
    for day in range(1, N_DAYS + 1):
        day_rows = by_day.get(day, [])
        rec = {
            "fecha": f"{YEAR}-{MONTH:02d}-{day:02d}",
            "horas_en_archivo": len(day_rows),
        }
        categories = {}
        for pol, (metric_col, _) in METRICS.items():
               metric_vals = [number(row.get(metric_col)) for _, row in
day_rows]
             metric_vals = [v for v in metric_vals if v is not None and v
>= 0]
                # Las partículas se promedian durante 24 h; gases usan
máximo horario o móvil de 8 h.
            minimum = 18
            if len(metric_vals) < minimum:
```

```
                indicator = None
            elif pol in ("PM10", "PM2.5"):
                indicator = sum(metric_vals) / len(metric_vals)
            else:
                indicator = max(metric_vals)
            indicator = round_base(pol, indicator)
             cat = category_for(pol, indicator) if indicator is not None
else None
            categories[pol] = cat
            rec[f"{pol}_n_valores_validos"] = len(metric_vals)
            rec[f"{pol}_indicador_diario"] = indicator
            rec[f"{pol}_categoria_diaria"] = cat or "ND"
        valid = [(p, c) for p, c in categories.items() if c]
          daily_category = max((c for _, c in valid), key=RANK.get) if
valid else None
        daily_rank = RANK.get(daily_category, -1)
         responsible = [p for p, c in valid if RANK[c] == daily_rank] if
daily_category else []
        rec["categoria_diaria"] = daily_category or "ND"
              rec["contaminantes_responsables"] = "+".join(p for p in
POLLUTANTS if p in responsible) or "ND"
        out.append(rec)
    return out

def export_hourly_csv(source, headers, records):
    path = BASE / "CEN_marzo_datos_procesados_export.csv"
    def write(target):
        with target.open("w", newline="", encoding="utf-8-sig") as f:
            w = csv.writer(f)
            w.writerow(headers)
             w.writerows([row.get(h, "") for h in headers] for _, row in
records)
    try:
        write(path)
    except PermissionError:
                                            path = BASE /
"CEN_marzo_datos_procesados_export_actualizado.csv"
        write(path)
    return path
```

```
def export_daily_csv(rows):
    path = BASE / "numeralia_diaria_CEN_marzo2024.csv"
    def write(target):
        with target.open("w", newline="", encoding="utf-8-sig") as f:
            w = csv.DictWriter(f, fieldnames=list(rows[0]))
            w.writeheader()
            w.writerows(rows)
    try:
        write(path)
    except PermissionError:
        path = BASE / "numeralia_diaria_CEN_marzo2024_actualizado.csv"
        write(path)
    return path

def write_summary(rows, source_name):
    valid_days = [r for r in rows if r["categoria_diaria"] != "ND"]
    cats = Counter(r["categoria_diaria"] for r in valid_days)
            dom   =   Counter(p   for   r   in   valid_days   for   p   in
r["contaminantes_responsables"].split("+") if p in POLLUTANTS)
     worst_rank = max((RANK[r["categoria_diaria"]] for r in valid_days),
default=-1)
     worst_days = [r["fecha"] for r in valid_days if worst_rank >= 0 and
RANK[r["categoria_diaria"]] == worst_rank]
    lines = [
        f"Numeralia diaria | Estación CEN | Marzo {YEAR}",
        f"Fuente procesada: {source_name}",
              f"Total  de  días:  {N_DAYS};  días  con  categoría  global:
{len(valid_days)}; días ND: {N_DAYS-len(valid_days)}.",
         "Días por categoría: " + ", ".join(f"{c}: {cats.get(c, 0)}" for
c in CATEGORIES),
          "Días como contaminante responsable (empates cuentan para cada
uno): " + ", ".join(f"{p}: {dom.get(p, 0)}" for p in POLLUTANTS),
              f"Categoría  más  desfavorable:  {CATEGORIES[worst_rank]  if
worst_rank  >=  0  else  'ND'};  "  +  (",  ".join(worst_days)  if  worst_days
else "sin días calculables"),
                    "Categoría   y   responsable   por   día:   "   +   ";
".join(f"{r['fecha'][-2:]}:                        {r['categoria_diaria']}
({r['contaminantes_responsables']})" for r in rows),
```

```
        "Método de resumen: promedio diario 24 h para PM10/PM2.5; máximo horario para O3/NO2/SO2; máximo de CO_prom_8h para CO. La categoría diaria es la peor entre los indicadores calculables.",
            "PM10 y PM2.5 requieren al menos 18 mediciones horarias; O3/NO2/SO2 y CO requieren al menos 18 valores del indicador diario usado por este análisis.",
          "Las columnas Categoria_Global y Categoria_* horarias de la fuente no se usan para esta agregación; sirven para una lectura de la evolución horaria.",
        "NA/ND en un contaminante indica que la hoja no proporcionó una categoría/medición válida.",
    ]
    path = BASE / "resumen_CEN_marzo2024.txt"
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")
    return path

def main():
    source, headers, records = read_source()
    daily = summarize(records, headers)

    # Genera únicamente los CSV y el reporte de texto
    outputs = [export_hourly_csv(source, headers, records),
               export_daily_csv(daily),
               write_summary(daily, source.name)]

    print(f"Fuente: {source.name} ({len(records)} filas horarias)")
    print("Salidas generadas:")
    for path in outputs:
        if path is not None: print(" -", path.name)

    print("\nResumen rápido:")
     print("Días por categoría:", dict(Counter(r["categoria_diaria"] for r in daily)))
                          print("Dominancia:",      {p:      sum(p      in r["contaminantes_responsables"].split("+")  for  r  in  daily)  for  p  in POLLUTANTS})

if __name__ == "__main__":
    main()
```

Como adición, se adjuntan los enlaces a los cuadernos del Colab donde se encuentran los códigos, ya que estos no fueron cargados a la plataforma de Github:

- Evaluación horaria: https://colab.research.google.com/drive/1tx969rMuNdw6T0iCy8WLheAXvNF28wdD?usp=sharing
- Evaluación diaria: https://colab.research.google.com/drive/1cdoBJylt9siDecXxBnpVPkWmfbCk2ANt#scrollTo=TxgEUjI79HGd

Declaración de IA generativa y tecnologías asistidas por IA

Durante la preparación de este trabajo, el autor o los autores utilizaron Gemini para la investigación de conceptos claves para el entendimiento de la actividad, tales como las variables y unidades medidas por la estación "Centro" del Sistema de Monitoreo Atmosférico de Jalisco y apoyar en la estructuración del reporte. Después de utilizar esta herramienta/servicio, el autor o los autores revisaron y editaron el contenido según fuera necesario y asumen la total responsabilidad por el contenido de este trabajo.

Bibliografía

Espinoza, L., Congacha, A., & Díaz, J. (2023). Calidad de datos con Python: un enfoque práctico. Esprint Investigación, 2(2), 26-34. https://doi.org/10.61347/ei.v2i2.55

Secretaría de Medio Ambiente y Recursos Naturales. (25 de enero de 2024). Norma Oficial Mexicana NOM-172-SEMARNAT-2023, Lineamientos para la obtención y comunicación del índice de calidad del aire y riesgos a la salud. Diario Oficial de la Federación.

Reporte del líder

Integrante	Actividades realizadas	Cumplimiento	Observaciones
Manzo Mejía Susana Rubí	Scrum Master - Organización y división de actividades. - Apoyo en Documentation para comprobar la integridad del documento y realizar correcciones menores	Completo	Considero que no es necesario realizar observaciones de manera individual ya que en mi opinión, todo el equipo ha trabajado de una manera correcta, cumpliendo con sus actividades en tiempo y forma para la realización de la actividad.
Castellanos Zuñiga Ian Michael	Storyteller - Redacción enfocada a nuestra agencia (SIMAJ).	Completo	
Montero González José Antonio	Data Engineer - Descarga y limpieza de los datos para su análisis.	Completo	
Melendez González Ángel Leonel	Data Analyst - Implementación de los cálculos necesarios para las evaluaciones horarias y diarias.	Completo	
Anaya Valladolid Luis Rafael	Visualization Designer - Diseño y desarrollo de gráficos con los datos obtenidos para una mejor interpretación.	Completo	
Magaña Mojica Jesús Ismael	QA / Documentation - Verificación e implementación de las secciones desarrolladas en un	Completo	

	solo documento siguiendo las pautas y los lineamientos.		

El desempeño general del equipo fue bastante bueno, todos trabajamos y pusimos nuestro granito de arena para que el desarrollo del Sprint se concretara con creces.

Un aspecto a considerar para mejorar de cara al siguiente Sprint, y los Sprints que resten, es continuar mejorando nuestra organización y la calidad con la que realizamos nuestros trabajos, porque siempre puede haber un margen de mejora.